

Python for AI engineers



Maya Malamud
Class October 2025
Lecture #3

Today's agenda

- ✓ Dictionaries
- ✓ Containers
- ✓ Functions & Decorators
- ✓ Scope of variables
- ✓ List Comprehension
- ✓ Magic functions
- ✓ Error handling

Dictionary

```
# Create an empty dictionary:  
x = {}  
Or  
x = dict()
```

Create a dictionary explicitly with all its initial items

```
crime_and_punishment = {"Name" : "Crime and Punishment",  
                        "Author" : "Fyodor Dostoevsky",  
                        "Year": 1866,  
                        "Genre" : ["Philosophical novel", "Psychological fiction", "Crime  
fiction"],  
                        "My rating" : 4.5,  
                        "Audience rating" : 4.5}
```

- A dictionary stores key-value pairs of objects, separated by comma.
- A dictionary is defined using curly braces { }.
- The keys are immutable and appear at most once in the dictionary.
- The values can be any valid Python object.
- Keys could be a number, string, tuple etc.
- Values could be number, string, tuple, lists, dictionaries etc.



Dictionary examples (1)

```
a = dict(a=1,b=2,c=3,d=4)
a
{'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

Create a dictionary with function dict()

```
In [1]: # Creating a simple dictionary
dict1={101:'ram',
        102:'sham',
        103:'ravi'}

# printing the whole dictionary
print(dict1)

# accessing a value using its key
dict1[103]|

{101: 'ram', 102: 'sham', 103: 'ravi'}

Out[1]: 'ravi'
```

Create a dictionary explicitly with all its initial items

Dictionary examples (2)

```
students_grade = {}
```

Create a new empty dictionary

```
students_grade['00001234'] = 85
```

```
students_grade['00003579'] = 95
```

```
students_grade['00002468'] = 65
```

Set a grade for 3 students

```
students_grade = {'00001234':85, '00003579':95}
```

Recreate the same dictionary with only 2 of the students. The third student will no longer exist in it.

```
In [3]: students_grade["0002468"] = 77
```

Add the third student with a grade to the dictionary.

```
In [4]: students_grade
```

```
Out[4]: {'0001234': 85, '0003579': 95, '0002468': 77}
```

Operations on dictionary

Method	Description
<code>dict.get(key, default)</code>	For key <code>key</code> , returns value or default if key not in dictionary
<code>dict.setdefault(key, default)</code>	Similar to <code>get()</code> , but will set <code>dict[key] = default</code> if key is not already in dict
<code>dict.fromkeys(seq, value)</code>	Create a new dictionary with keys from <code>seq</code> and values set to <code>value</code> .
<code>dict.keys()</code>	Returns a view of dictionary <code>dict</code> 's keys
<code>dict.items()</code>	Returns a view of <code>dict</code> 's (key, value) tuple pairs
<code>dict.values()</code>	Returns a view of dictionary <code>dict</code> 's values
<code>dict.update(dict2)</code>	Adds dictionary <code>dict2</code> 's key-values pairs to <code>dict</code>
<code>dict.pop(key, [default])</code>	If <code>key</code> is in the dictionary, remove it and return its value, else return <i>default</i> . If <i>default</i> is not given and <code>key</code> is not in the dictionary, a <code>KeyError</code> is raised.

```
dict1={101:'ram',  
dict1.  
clear  
copy  
fromkeys  
get  
items  
keys  
pop  
popitem  
setdefault  
update
```

dict.setdefault()

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
  
x = car.setdefault("model", "Bronco")  
  
print(x)
```

Mustang

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
  
x = car.setdefault("color", "white")  
  
print(x)
```

white

```
[21] students = {'Alice': 21, 'Bob': 24, 'Charlie': 23}
```

```
age = students.setdefault('Alice', 22)  
print(age)
```

```
age = students.setdefault('David', 25)  
print(age)  
print(students)
```

21

25

{'Alice': 21, 'Bob': 24, 'Charlie': 23, 'David': 25}



dict.setdefault():

- If key exists, it will return the key's existing value.
- If the key doesn't exist, it will add the new key and its value to the dictionary.

Looping through dictionaries

By default, will print keys

```
for i in car:  
    print(i)
```

brand
model
year
color

Access values

```
for val in car.values():  
    print(val)
```

Ford
Mustang
1964
white

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
  
x = car.setdefault("color", "white")  
  
print(x)  
  
white
```

Get tuples of key, value

```
for item in car.items():  
    print(item)
```

('brand', 'Ford')
('model', 'Mustang')
('year', 1964)
('color', 'white')

Get key and value separately

```
for key, val in car.items():  
    print("the key is:", key + " " + "and the value is:", val)
```

the key is: brand and the value is: Ford
the key is: model and the value is: Mustang
the key is: year and the value is: 1964
the key is: color and the value is: white

Comma (,) will add a space and plus (+) won't

Containers

A Container is an object that is used to store different objects and provide a way to access the contained objects and iterate over them. Some of the built-in containers are Tuple, List, Dictionary, etc.

Looping through Containers:

- Enumerate
- Zip



Looping through Containers: Enumerate

The built-in function `enumerate` is a convenient way to access both the index of an item and the item itself when looping over a list or any iterable object.

```
fruits = ['apple', 'banana', 'cherry']
```

← List of Fruits

```
for index, item in enumerate(fruits):  
    print(index, item)
```

← Enumerate function
used to unpack the
index and value pair

```
0 apple  
1 banana  
2 cherry
```

Output

↑ Index ↑ Value

What is the input for this?

```
for count, value in enumerate(values):  
    print(count, value)
```

```
0 a  
1 b  
2 c
```

```
for i, v in enumerate(['tic', 'tac', 'toe']):  
    print(i, v)
```

```
0 tic  
1 tac  
2 toe
```

```
for i, v in enumerate(['tic', 'tac', 'toe'], start=1):  
    print(i, v)
```

```
1 tic  
2 tac  
3 toe
```

Looping through Containers: Zip

- Combines similar type iterators (list-list or dict- dict, etc) printing the values sequentially.
- Returns an iterator of tuples, where the i-th tuple contains the i-th element from each of the argument sequences or iterables.
- The iterator stops when the shortest input iterable is exhausted.

```
a=[1,2,3]
b = [4,5,6]
zip(a,b)
```

<zip at 0x2627c3e1408>

Will not print unless called
with list or loop
(same as range)

```
a=[1,2,3]
b = [4,5,6]
list(zip(a,b))
```

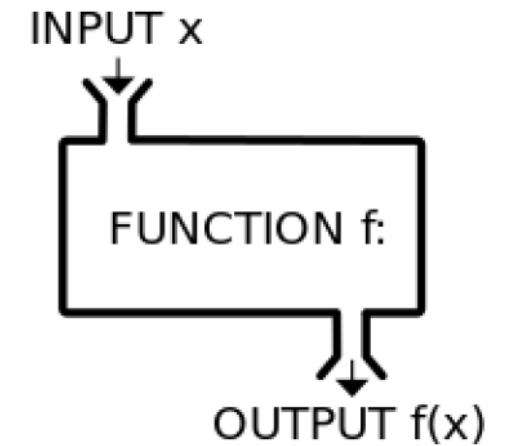
[(1, 4), (2, 5), (3, 6)]

```
questions = ['name', 'quest', 'favorite color']
answers = ['lancelot', 'the holy grail', 'blue']
for q, a in zip(questions, answers):
    print('What is your', q + "?", "It is", a)
```

What is your name? It is lancelot
What is your quest? It is the holy grail
What is your favorite color? It is blue

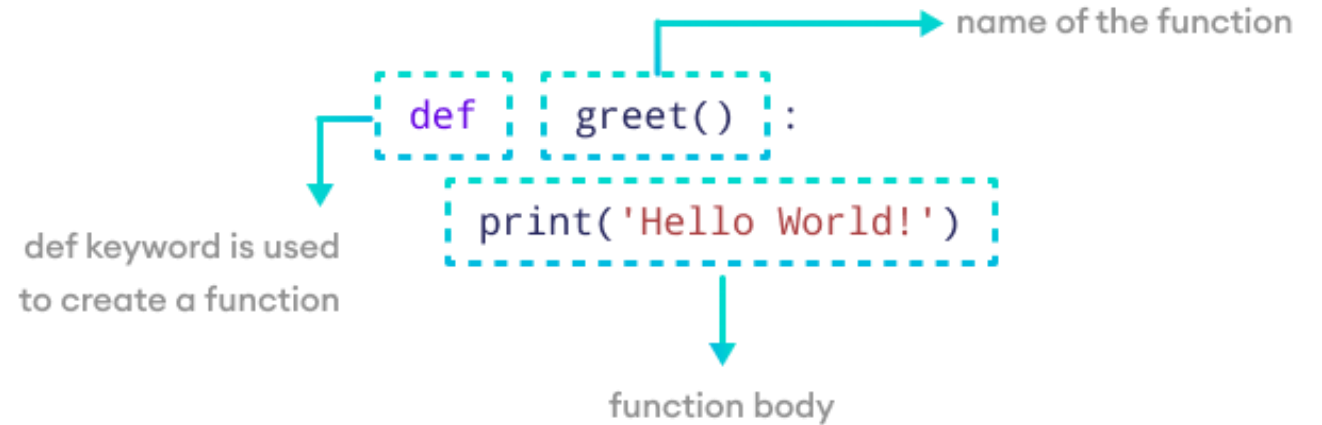
Functions – built-in functions

- A function is a structuring element in programming languages to group a set of statements so they can be utilized more than once in a program.
- Functions offer a way to:
 - Reuse and share code
 - Help organize the program
- Functions can receive arguments and return values.
- Python has few built-in functions.
- Other functions reside in libraries and can be added manually.
 - For example:
 - `int("40")` # *Converts string to integer number*
 - `str(40)` # *Converts number to string*



Function: how to

```
def <function_name> ([parameters]):  
    """function_docstring\documentation"""  
    <statement(s)>
```



Component	Meaning
<code>def</code>	The keyword that informs Python that a function is being defined
<code><function_name></code>	A valid Python identifier that names the function
<code><parameters></code>	An optional, comma-separated list of parameters that may be passed to the function
<code>:</code>	Punctuation that denotes the end of the Python function header (the name and parameter list)
<code><statement(s)></code> / Body	A block of valid Python statements

Function (help, doc string)

```
In [2]: def hello():  
...:     """this function only prints hello world"""  
...:     print("hello world")  
...:
```

```
In [3]: hello()  
hello world
```

```
In [4]: help(hello)  
Help on function hello in module __main__:  
  
hello()  
    this function only prints hello world
```

```
In [17]: def print_message(msg):  
...:     """print a message  
...:     Args:  
...:         msg must be a string"""  
...:     print(msg)  
...:
```

```
In [18]: print_message()  
Traceback (most recent call last):
```

```
File "<ipython-input-18-14ad73f3dd9d>", line 1, in <module>  
    print_message()
```

```
TypeError: print_message() missing 1 required positional argument: 'msg'
```

```
In [19]:
```

```
In [19]: print_message("danny")  
danny
```

Return statement

- It immediately terminates the function and passes execution control back to the caller.
- It provides a mechanism by which the function can pass data back to the caller.

```
def add_numbers():  
    x=2  
    y=4  
    print(x,y)  
    return(x+y)  
add_numbers()
```

2 4

6

```
def print_my_name():  
    return "danny"  
    print("hello world")
```

```
print("hello", print_my_name())
```

Return statement examples

A function can print out the result or return it.

```
In [25]: def seven():  
...:     "return the number 7"  
...:     x = 7  
...:     return x  
...:
```

```
In [26]: seven()
```

```
Out[26]: 7
```

Change the add_number() function into argument/ parameter form. Return only the sum!

```
def add_number(x, y):  
    "Sum 2 numbers, x and y must be int"  
    return x+y  
sum_num = add_number(2, 5)  
print(sum_num) → 7
```

```
In [27]: def add_number():  
...:     "sum 2 numbers"  
...:     x = 7  
...:     y = 35  
...:     return x,y,x+y  
...:
```

```
In [27]:
```

```
In [28]: add_number()
```

```
Out[28]: (7, 35, 42)
```

```
In [29]: type(add_number)
```

```
Out[29]: function
```

```
In [30]: type(add_number())
```

```
Out[30]: tuple
```


Return statement: None

```
def stam():  
    return None  
k = stam()  
print(k)
```

```
def f():  
    print("foo")  
    print("bar")  
    return  
f()
```

foo
bar

```
In [51]: def f(x):  
        if x < 0:  
            return  
        if x > 100:  
            return  
        print(x)
```

```
In [52]: f(-3)
```

```
In [53]: f(125)
```

```
In [54]: f(64)
```

64

Functions

Default argument / parameter values

```
def add_numbers(n1 = 100, n2 = 1000):  
    sum = n1 + n2  
    return sum  
  
result = add_numbers()  
print(result)
```

Call the function without arguments

```
def add_numbers(n1 = 100, n2 = 1000):  
    result = n1 + n2  
    return result  
  
result = add_numbers(5.4)  
print(result) #1005.4
```

Call the function with only 1 argument

```
def area_triangle(a, b, area=2):  
    return(a*b *(1/area))  
print(area_triangle(4,5))  
print(area_triangle(4,5,3))
```

10.0

6.6666666666666666

Both default and non-default arguments

```
def area_triangle(a=3, b, area=2):  
    return(a*b *(1/area))  
print(area_triangle(4,5))  
print(area_triangle(4,5,3))
```

File "<ipython-input-46-4ff9336dbbd7>", line 1

def area_triangle(a=3, b, area=2):

SyntaxError: non-default argument follows default argument

You should declare default arguments after non-default arguments

*args

```
def foo(*args):  
    print(args)  
foo(1,2,3)
```

(1, 2, 3)

```
def my_sum(*integers):  
    result = 0  
    for x in integers:  
        result += x  
    return result
```

```
my_sum(1, 2, 3)
```

6

- * Is the unpacking operator
- Pass an unspecified number of arguments using *args
- The name doesn't have to be "args" but it's the conventional name
- This is useful for a varying number of arguments passed to the function at each call.

****kwargs (keyword arguments)**

****kwargs** works just like ***args**, but instead of accepting positional arguments it accepts keyword (or named) arguments.

```
def my_function(**Patient):  
    print("Her last name is " + Patient["lname"])  
    print(type(Patient))
```

```
my_function(fname = "Maya", lname = "Malamud")
```

```
Her last name is Malamud  
<class 'dict'>
```

Similar to ***args**, ****kwargs** allows us to handle named arguments that we have not defined in advance.

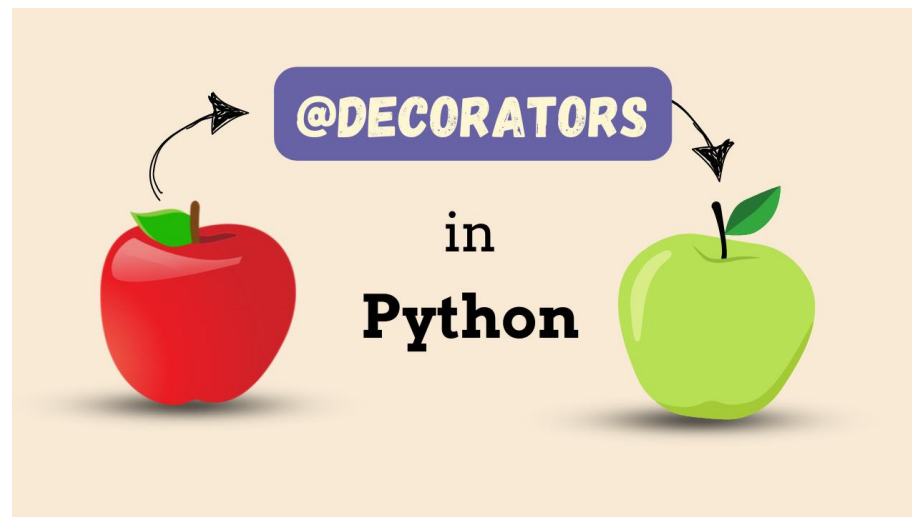
```
def concatenate(**kwargs):  
    result = ""  
    # Iterating over the Python kwargs dictionary  
    for arg in kwargs.values():  
        result += arg  
    return result  
  
concatenate(a="Python", b="Is", c="Great", d="!")
```

Output: 'PythonIsGreat!'

For output 'abcd' we need `kwargs.keys()`

Decorators

- Decorators are a powerful and flexible way to modify or extend the behavior of functions or methods, without changing their actual code.
- It's essentially a function that takes another function as an argument and returns a new function with enhanced functionality.
- Often used in scenarios such as logging, authentication and memorization.
- Allowing us to add additional functionality to existing functions or methods in a clean, reusable way.



Decorators

```
# A simple decorator function
def decorator(func):

    def wrapper():
        print("Before calling the function.")
        func()
        print("After calling the function.")
    return wrapper

# Applying the decorator to a function
@decorator
def greet():
    print("Hello, World!")

greet()
```

Before calling the function.
Hello, World!
After calling the function.

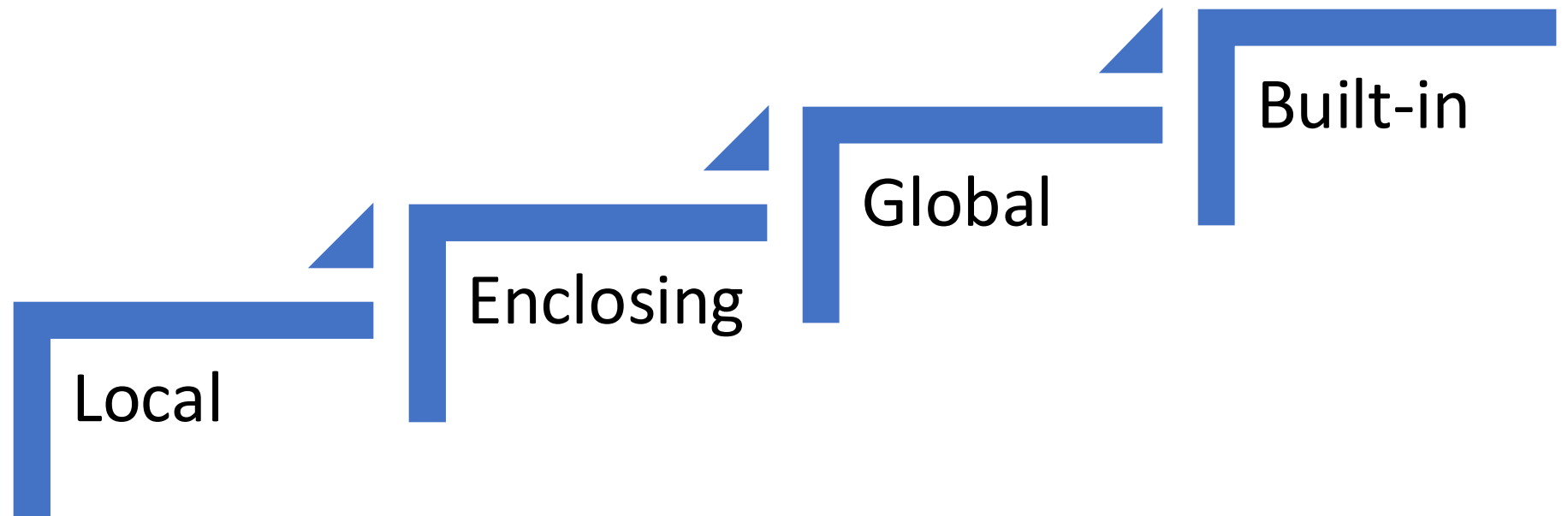
- Decorator takes the greet function as an argument.
- It returns a new function (wrapper) that first prints a message, calls greet() and then prints another message.

Scope of variables

Local, Enclosing, Global, Built-in

Where does our variable exist?

LEGB



Scope of variables

Local scope

Local: Whenever you define a variable within a function, its scope lies ONLY within the function

```
n [3]: def print_number():  
        first_num = 1  
        # Print statement 1  
        print("The first number defined is: ", first_num)
```

```
n [4]: print_number()
```

The first number defined is: 1

```
n [5]: print("The first number defined is: ", first_num)
```

```
-----  
NameError                                Traceback (most recent call last)  
<ipython-input-5-4c177df29e75> in <module>  
----> 1 print("The first number defined is: ", first_num)
```

NameError: name 'first_num' is not defined

```
x, y = 0,1  
def increment(x):  
    y= x+1  
    return(y)  
print("Original X and Y:", x,y)  
print("function value:", increment(5))
```

Original X and Y: 0 1
function value: 6

Scope of variables

Enclosing non-local scope

LEGB (Local -> Enclosing -> Global -> Built-in)

Enclosing scope is a special scope that exists only for a nested function

```
: inner_func()
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-15-1d4cd1e06103> in <module>
----> 1 inner_func()
```

NameError: name 'inner_func' is not defined

```
def outer_func():
    # This block is the Local scope of outer_func()
    first_num = 1 # a nonlocal variable
    def inner_func():
        # This block is the Local scope of inner_func()
        second_num = 2
        # Print statement 1 - Scope: Inner
        print("first_num from outer: ", first_num)
        # Print statement 2 - Scope: Inner
        print("second_num from inner: ", second_num)
    inner_func()
    # Print statement 3 - Scope: Outer
    print("second_num from inner: ", second_num)
```

```
outer_func()
```

```
first_num from outer:  1
second_num from inner: 2
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-14-be2031892881> in <module>
----> 1 outer_func()

<ipython-input-13-6378b03f919e> in outer_func()
     11     inner_func()
     12     # Print statement 3 - Scope: Outer
----> 13     print("second_num from inner: ", second_num)
```

NameError: name 'second_num' is not defined

Scope of variables

Global module scope

LEGB (Local -> Enclosing -> Global -> Built-in)

Whenever a variable is defined outside any function, it becomes a global variable, and its scope is anywhere within the program. Which means it can be used by any function.

```
greeting = "Hello"

def greeting_world():
    world = "World"
    print(greeting, world)

def greeting_name(name):
    print(greeting, name)

greeting_world()
greeting_name("Samuel")
```

Hello World
Hello Samuel

```
var = 100
def func():
    var = 20
    return var
print(func())
print(var)
```

20
100

```
var = 100
def change_var():
    global var
    var = 30
    return var
print(change_var())
print(var)
```

30
30

Scope of variables

Built-in Scope

LEGB (Local -> Enclosing -> Global -> Built-in)

The widest scope that exists!

- All keywords / reserved words that we can call anywhere within our program without having to define them before use

```
def print(str):  
    pass  
  
print('hi')
```

Defining a function print,
It replaces Python's print function!

- Not recommended!

Scope of variables

LEGB rule

LEGB (Local -> Enclosing -> Global -> Built-in)

- The LEGB rule is a kind of name lookup procedure, which determines the order in which Python looks up names.
- For example, if we access a name, then Python will look that name up sequentially in the local, enclosing, global, and built-in scope.



List comprehension

- List comprehension is an elegant way to define and create lists in Python
 - a one liner 😊

`new_list = [expression for member in iterable]`

```
cashier_3 = []  
for item in cart:  
    if item % 2 == 0:  
        cashier_3.append(item)
```

Non-list comprehension

```
cashier_3 = [item for item in cart if item % 2 == 0]
```

List comprehension

List comprehension: examples

```
squared = []  
for i in range(10):  
    squared.append(i**2)  
squared
```

[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

```
squared = []  
for i in range(10):  
    if i%2==0:  
        squared.append(i**2)  
squared
```

[0, 4, 16, 36, 64]

```
squared_list_comp = [i**2 for i in range(10)]  
squared_list_comp
```

[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

```
[i**2 for i in range(10) if i%2==0]
```

[0, 4, 16, 36, 64]

Sorted Function

- The built-in function `sorted(iterable, key, reverse)` builds a new sorted list from an iterable.
- It has two optional parameters:
 1. `key`, receives a function that determines the comparison key
 2. `reverse`, which sorts the list in a reversed order

```
sorted([8,3,4,1,2])
```

```
[1, 2, 3, 4, 8]
```

```
sorted([2,34,56,8,9,0], reverse=True)
```

```
[56, 34, 9, 8, 2, 0]
```

```
dct = {1: 'D', 2: 'B', 3: 'B', 4: 'E', 5: 'A'}
```

Sorted Function with dictionary

```
dct = {1: 'D', 2: 'B', 3: 'B', 4: 'E', 5: 'A'}  
sorted(dct)
```

```
[1, 2, 3, 4, 5]
```

Returns list of dictionary's keys

```
sorted(dct.items())
```

```
[(1, 'D'), (2, 'B'), (3, 'B'), (4, 'E'), (5, 'A')]
```

Sort items by dictionary keys

```
sorted(dct.values())
```

```
['A', 'B', 'B', 'D', 'E']
```

Sort dictionary values

```
sorted(dct, key=dct.get)
```

```
[5, 2, 3, 1, 4]
```

Sort keys by dictionary values

Sorted Function examples

```
sorted("This is a test string from Andrew".split())
```

```
['Andrew', 'This', 'a', 'from', 'is', 'string', 'test']
```

```
sorted("This is a test string from Andrew".split(), key= str.lower)
```

```
['a', 'Andrew', 'from', 'is', 'string', 'test', 'This']
```

```
def get_key(item):  
    return item[0]  
l = [[2, 3], [6, 7], [3, 34], [24, 64], [1, 43]]  
sorted(l, key=get_key)  
[[1, 43], [2, 3], [3, 34], [6, 7], [24, 64]]  
  
[[1, 43], [2, 3], [3, 34], [6, 7], [24, 64]]
```

```
def get_key(item):  
    return item[1]  
student_tuples = [('john', 'A', 15), ('jane', 'B', 12), ('dave', 'B', 10)]  
sorted(student_tuples, key=get_key)  
  
[('john', 'A', 15), ('jane', 'B', 12), ('dave', 'B', 10)]
```

```
sorted("encyclopedia")
```

```
['a', 'c', 'c', 'd', 'e', 'e', 'i', 'l', 'n', 'o', 'p', 'y']
```

Magic functions: Lambda

- Lambda functions are single-line functions defined without a name (also called anonymous functions).
- Usually used for a one-time non-complicated function.
- Syntax: `lambda arg1, arg2, ... : expression to return`

```
def f(x):  
    return 2*x+1
```

```
f = lambda x: 2*x+1
```

Call f like a regular function: `f(5) → 11`

```
new_f = lambda x,y: x*y + x + y  
new_f(2,3)
```

Lambda example

```
def get_key(item):  
    return item[1]  
student_tuples = [('john', 'A', 15), ('jane', 'B', 12), ('dave', 'B', 10)]  
sorted(student_tuples, key=get_key)
```

```
[('john', 'A', 15), ('jane', 'B', 12), ('dave', 'B', 10)]
```

```
student_tuples = [('john', 'A', 15), ('jane', 'B', 12), ('dave', 'B', 10)]  
sorted(student_tuples, key=lambda student: student[2])
```

```
[('dave', 'B', 10), ('jane', 'B', 12), ('john', 'A', 15)]
```

Magic functions: map

- Gets a function and an iterable (such as list)
- Calls the function for each element in the iterable
- Returns the iterable after applying the function to each of its elements

Long way

```
new_list = []  
for num in range(10):  
    new_list.append(cube(num))  
new_list
```

```
[0, 1, 8, 27, 64, 125, 216, 343, 512, 729]
```

Short way
using map and function

```
def cube(x):  
    return x*x*x
```

```
list(map(cube, range(10)))
```

```
[0, 1, 8, 27, 64, 125, 216, 343, 512, 729]
```

Short way
using map and lambda

```
lst = range(10)  
list(map(lambda x: x*x, lst))
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Magic functions: filter

- Gets a function and an iterable (such as string)
- Calls the function for each element in the iterable
- Returns the iterable after filtering in only the elements for which the function returned True

```
def filter_vowels(variable):  
    letters = ['a', 'e', 'i', 'o', 'u']  
    if (variable in letters):  
        return True  
    else:  
        return False  
  
#sequence = ['g', 'e', 'e', 'j', 'k', 's', 'p', 'r']  
sequence = "encyclopedia"  
list(filter(filter_vowels, sequence))
```

```
['e', 'o', 'e', 'i', 'a']
```

using lambda as the function

```
list(filter(lambda x: x % 2 != 0, range(10)))
```

```
[1, 3, 5, 7, 9]
```

Magic functions: reduce

- Reduce applies a rolling computation to sequential pairs of values in a list.
- Very useful function for performing some computation on a list and returning the result.

```
product = 1
lst = [1, 2, 3, 4]
for num in lst:
    product = product * num
product
```

24

```
* [1, 2, 3, 4] #2*1
* [2, 3, 4] #2*3
* [6, 4] # 6*4
* [24]
```

```
from functools import reduce
product = reduce((lambda x, y: x * y), lst)
product
```

24

- At first step, first two elements of sequence are picked, and the result is obtained.
- Next step is to apply the same function to the previously attained result and the number just succeeding the second element and the result is again stored.
- This process continues till no more elements are left in the container.
- The final returned result is returned and printed on console.

Error handling

Error examples

```
print(word)
```

```
-----  
NameError                                Tr  
<ipython-input-13-60d3f5ba92a8> in <module>  
----> 1 print(word)  
  
NameError: name 'word' is not defined
```

error: printing un-defined variable

```
st = "python"  
st[1]="d"
```

```
-----  
TypeError                                Traceback (most  
<ipython-input-12-5ac8bde26e6d> in <module>  
      1 st = "python"  
----> 2 st[1]="d"  
  
TypeError: 'str' object does not support item assignment
```

error: assigning item to a string location

```
"1234"+26
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-11-20dcf67c2704> in <module>  
----> 1 "1234"+26  
  
TypeError: can only concatenate str (not "int") to str
```

error: concatenating int to string

Syntax error vs. Exception

- Syntax Error: Python cannot start to execute your code

```
while True print "gello world"
```

```
File "<ipython-input-59-b00c9b7924bd>", line 1
```

```
while True print "gello world"
```

^

SyntaxError: invalid syntax

Syntax error: missing print parenthesis

```
for num in range(5):  
print(num)
```

```
File "<ipython-input-61-a2f739a6013f>", line 2
```

```
print(num)
```

^

IndentationError: expected an indented block

Syntax error: missing indentation for code within function

- Exception: logical error occurring in correct code blocks at run time
 - For example, dividing a number by zero.
 - Opening a file that doesn't exist.

Exceptions handling

Exception handling is the process of handling exceptions in a custom way (rather than showing the default error message).

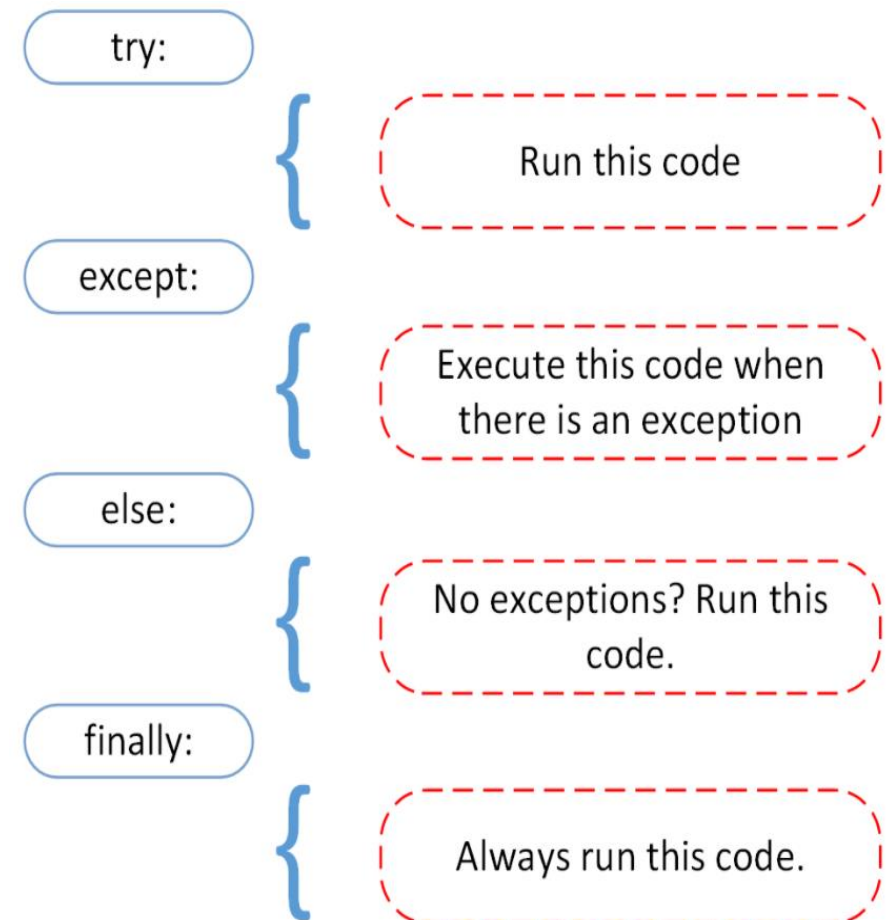
“Try” block: contains the code to be monitored for the exceptions, terminated when the first exception occurs.

“Except” block: contains what to be done if a specific exception occurs.

- If no exception occurs, then this section is skipped
- If an exception occurs, catch it with except

“Else” block (optional): executed only if there is no exception in a try block. If any exception is raised, the else block will not be executed.

“Finally” block (optional): always executed.



Exceptions handling

try, except

It's helpful to use try and except when checking for correctness of input, such as for arguments of a function

```
def sum_str_num(string, num):  
    return string+num  
sum_str_num("1", 4)
```

```
-----  
TypeError                                 Traceback (most  
<ipython-input-21-4d9fb1e276b8> in <module>  
      1 def sum_str_num(string, num):  
      2     return string+num  
----> 3 sum_str_num("1", 4)  
  
<ipython-input-21-4d9fb1e276b8> in sum_str_num(string, num  
      1 def sum_str_num(string, num):  
----> 2     return string+num  
      3 sum_str_num("1", 4)
```

TypeError: can only concatenate str (not "int") to str

```
def sum_str_num(string, num):  
    try:  
        return string+num  
    except:  
        print("Error: Try again")  
sum_str_num("1", 4)
```

Error: Try again

Exceptions handling

try, except instead of if-else

```
def sum_str_num_conditioned(string, num):  
    if type(string)==str and type(num)==str:  
        return f'concat two string: {string+num}'  
    elif type(string)==int and type(num)==int:  
        return f'concat two integer numbers: {string+num}'  
    elif type(string)==int and type(num)==float:  
        return f'concat integer and float numbers: {string+num}'  
    elif type(string)==float and type(num)==int:  
        return f'concat integer and float numbers: {string+num}'  
    elif type(string)==float and type(num)==float:  
        return f'concat two float numbers: {string+num}'  
    elif type(string)==str and type(num)==int:  
        return 'Error'  
print(sum_str_num_conditioned(2,2))  
print(sum_str_num_conditioned(0.45,2))  
print(sum_str_num_conditioned("d",2))
```

concat two integer numbers: 4
concat integer and float numbers: 2.45
Error

```
def do_something(thing):  
    try:  
        print("Hello" + thing)  
    except:  
        print("Error")
```

Upside: shorter and cleaner code

Downside: the error isn't informative

```
def do_something(thing):  
    x = 0  
    try:  
        x = 1  
        print("Hello" + thing)  
        x = 2  
        print(x)  
    except:  
        print(x)
```

The printed x will be 1 in case of except,
code before the error is executed

Exceptions handling customized error message

```
def sum_str_num(string, num):  
    try:  
        return string+num  
    except Exception as e:  
        print(e)
```

```
sum_str_num("d", 2)
```

can only concatenate str (not "int") to str

Print the exception,
meaning the error message

```
def divide_numbers(num1, num2):  
    return num1/num2  
divide_numbers(2/0)
```

ZeroDivisionError

```
<ipython-input-54-ac536b354ec3> in <mod  
      1 def divide_numbers(num1, num2):  
      2     return num1/num2  
----> 3 divide_numbers(2/0)
```

ZeroDivisionError: division by zero

The name of the exception: ZeroDivisionError

```
def do_something_that_will_raise_exception(num):  
    return num/0  
  
num = 12  
try:  
    do_something_that_will_raise_exception(num)  
except ZeroDivisionError:  
    print("Zero Division Error")  
except TypeError:  
    print("Type Error")  
except Exception as e:  
    print(e)
```

Zero Division Error

Print the exception by 2 categories or all others

Class exercise – Lecture 3

Mini Bookstore Data Tool - You're helping a small online bookstore analyze and manage its inventory. You'll use Python to write functions, work with containers, and apply key programming tools.

1. Create a **list of dictionaries**, each representing a book:

```
books = [ {"title": "Python 101", "price": 39.99, "rating": 4.5},  
          {"title": "Deep Learning", "price": 59.99, "rating": 4.8}, {"title":  
"AI for Beginners", "price": 29.99, "rating": 4.2}, {"title": "Data  
Science Daily", "price": 49.99, "rating": 3.9}, {"title": "The Bug  
Book", "price": 19.99, "rating": 3.2} ]
```

2. Loop through each dictionary and print key-value pairs.

3. Functions:

- Write a function `is_highly_rated(rating)` that returns True if rating ≥ 4.5 .
- Write a function `get_top_books(book_list)` that uses the first function to filter top-rated books.
- Try printing rating outside the function. What happens? Why?

4. Use the `sorted()` function to sort Books by rating (high to low) and print the top 3 results.

5. Magic functions:

- Use `map()` with a lambda to round all book prices to the nearest integer.
- Use `filter()` to keep books under \$40.

6. Error Handling: Create a function `calculate_discount(price)` that divides 100 by price.

- Use a try/except block to catch a **ZeroDivisionError** and print a helpful message.
- Test it with a fake book with price 0.

7. Bonus - Decorators:

- Write a decorator `@log_call` that prints "Running function..." before a function executes.
- Use it to decorate `get_top_books()` or `calculate_discount()`.

Class Solution – Lecture 3

Mini Bookstore Data Tool - You're helping a small online bookstore analyze and manage its inventory. You'll use Python to write functions, work with containers, and apply key programming tools.

```
# 1. Create a list of book dictionaries
books = [
    {"title": "Python 101", "price": 39.99, "rating": 4.5},
    {"title": "Deep Learning", "price": 59.99, "rating": 4.8},
    {"title": "AI for Beginners", "price": 29.99, "rating": 4.2},
    {"title": "Data Science Daily", "price": 49.99, "rating": 3.9},
    {"title": "The Bug Book", "price": 19.99, "rating": 3.2}
]

# 2. Loop through and print key-value pairs
print("Book Details:")
for book in books:
    for key, value in book.items():
        print(f"{key}: {value}")
    print("---")

# 3. Functions
def is_highly_rated(rating):
    return rating >= 4.5

def get_top_books(book_list):
    return [book for book in book_list if
            is_highly_rated(book["rating"])]

# Try to access rating (defined in function scope only)
try:
    print(rating)
except NameError as e:
    print("Error:", e) # Variable 'rating' is not defined outside
the function
```

```
# 4. Sorted books by rating, descending
books_sorted_by_rating = sorted(books, key=lambda x: x["rating"],
reverse=True)
print("\nTop 3 Books by Rating:")
for book in books_sorted_by_rating[:3]:
    print(f"{book['title']} – Rating: {book['rating']}")

# 5. Magic Functions
# map() with lambda to round prices
rounded_prices = list(map(lambda b: round(b["price"]), books))
print("\nRounded Prices:", rounded_prices)

# filter() to get affordable books
affordable_books = list(filter(lambda b: b["price"] < 40, books))
print("Books under $40:", [book["title"] for book in
affordable_books])

# 6. Error handling
def calculate_discount(price):
    try:
        return 100 / price
    except ZeroDivisionError:
        return "Error: Price cannot be zero!"
```

```
# Add a fake book with price 0
fake_book = {"title": "Broken Book", "price": 0, "rating": 1.0}
books.append(fake_book)

# Test discount
print("\nDiscount test:")
for book in books[-2:]: # just last two
    print(f"{book['title']}: {calculate_discount(book['price'])}")

# 7. Bonus - Decorator
def log_call(func):
    def wrapper(*args, **kwargs):
        print(f"Running function: {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_call
def get_top_books_logged(book_list):
    return [book for book in book_list if
            is_highly_rated(book["rating"])]

print("\nTop Rated Books (using decorated function):")
for book in get_top_books_logged(books):
    print(book["title"])
```